

数据结构代码题

17. 【2009 统考真题】已知一个带有表头结点的单链表，结点结构为

data	link
------	------

假设该链表只给出了头指针 `list`。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点 (k 为正整数)。若查找成功，算法输出该结点的 `data` 域的值，并返回 1；否则，只返回 0。要求：

- 1) 描述算法的基本设计思想。
- 2) 描述算法的详细实现步骤。
- 3) 根据设计思想和实现步骤，采用程序设计语言描述算法（使用 C、C++ 或 Java 语言实现），关键之处请给出简要注释。

17. 【解答】

1) 算法的基本设计思想如下:

问题的关键是设计一个尽可能高效的算法, 通过链表的一次遍历, 找到倒数第 k 个结点的位置。算法的基本设计思想是: 定义两个指针变量 p 和 q , 初始时均指向头结点的下一个结点 (链表的第一个结点), p 指针沿链表移动; 当 p 指针移动到第 k 个结点时, q 指针开始与 p 指针同步移动; 当 p 指针移动到最后一个结点时, q 指针所指示结点为倒数第 k 个结点。以上过程对链表仅进行一遍扫描。

2) 算法的详细实现步骤如下:

- ① $count=0$, p 和 q 指向链表表头结点的下一个结点。
- ② 若 p 为空, 转⑤。
- ③ 若 $count$ 等于 k , 则 q 指向下一个结点; 否则, $count=count+1$ 。
- ④ p 指向下一个结点, 转②。
- ⑤ 若 $count$ 等于 k , 则查找成功, 输出该结点的 $data$ 域的值, 返回 1; 否则, 说明 k 值超过了线性表的长度, 查找失败, 返回 0。
- ⑥ 算法结束。

3) 算法实现如下:

```
typedef int ElemType;           //链表数据的类型定义
typedef struct LNode{           //链表结点的结构定义
    ElemType data;              //结点数据
    struct LNode *link;         //结点链接指针
}LNode, *LinkList;

int Search_k(LinkList list,int k){
    LNode *p=list->link,*q=list->link; //指针 p、q 指示第一个结点
    int count=0;
    while(p!=NULL){              //遍历链表直到最后一个结点
        if (count<k) count++;      //计数, 若 count<k 只移动 p
        else q=q->link;
        p=p->link;                //之后让 p、q 同步移动
    }//while
    if(count<k)
        return 0;                //查找失败返回 0
    else {
        printf("%d",q->data);      //否则打印并返回 1
        return 1;
    }
}
```

评分说明: 若所给出的算法采用一遍扫描方式就能得到正确结果, 可给满分 15 分; 若采用两遍或多遍扫描才能得到正确结果, 最高分为 10 分。若采用递归算法得到正确结果, 最高给 10 分; 若实现算法的空间复杂度过高 (使用了大小与 k 有关的辅助数组), 但结果正确, 最高给 10 分。

- 
10. 【2010 统考真题】设将 n ($n > 1$) 个整数存放于一维数组 R 中。设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p ($0 < p < n$) 个位置, 即将 R 中的数据由 $(X_0, X_1, \dots, X_{n-1})$ 变换为 $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ 。要求:
- 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 或 Java 语言描述算法, 关键之处给出注释。
 - 3) 说明你所设计算法的时间复杂度和空间复杂度。



10. 【解答】

1) 算法的基本设计思想:

可将问题视为把数组 ab 转换成数组 ba (a 代表数组的前 p 个元素, b 代表数组中余下的 $n-p$ 个元素), 先将 a 逆置得到 $a^{-1}b$, 再将 b 逆置得到 $a^{-1}b^{-1}$, 最后将整个 $a^{-1}b^{-1}$ 逆置得到 $(a^{-1}b^{-1})^{-1} = ba$ 。设 Reverse 函数执行将数组逆置的操作, 对 abcdefgh 向左循环移动 3 ($p=3$) 个位置的过程如下:

Reverse(0, $p-1$) 得到 cbadefgh;

Reverse(p , $n-1$) 得到 cbahgfed;

Reverse(0, $n-1$) 得到 defghabc。

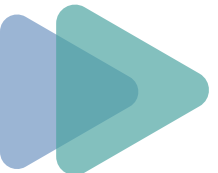
注: 在 Reverse 中, 两个参数分别表示数组中待转换元素的始末位置。

2) 使用 C 语言描述算法如下:

```
void Reverse(int R[],int from,int to) {
    int i,temp;
    for(i=0;i<(to-from+1)/2;i++)
        {temp=R[from+i];R[from+i]=R[to-i];R[to-i]=temp;}
}
void Converse(int R[],int n,int p){
    Reverse(R,0,p-1);
    Reverse(R,p,n-1);
    Reverse(R,0,n-1);
}
```

3) 上述算法中三个 Reverse 函数的时间复杂度分别为 $O(p/2)$ 、 $O((n-p)/2)$ 和 $O(n/2)$, 故所设计的算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

【另解】 借助辅助数组来实现。算法思想: 创建大小为 p 的辅助数组 S , 将 R 中前 p 个整数依次暂存在 S 中, 同时将 R 中后 $n-p$ 个整数左移, 然后将 S 中暂存的 p 个数依次放回到 R 中的后续单元。时间复杂度为 $O(n)$, 空间复杂度为 $O(p)$ 。



11. 【2011 统考真题】一个长度为 L ($L \geq 1$) 的升序序列 S , 处在第 $\lceil L/2 \rceil$ 个位置的数称为 S 的中位数。例如, 若序列 $S_1 = (11, 13, 15, 17, 19)$, 则 S_1 的中位数是 15, 两个序列的中位数是含它们所有元素的升序序列的中位数。例如, 若 $S_2 = (2, 4, 6, 8, 20)$, 则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B , 试设计一个在时间和空间两方面都尽可能高效的算法, 找出两个序列 A 和 B 的中位数。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 或 Java 语言描述算法, 关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。



11. 【解答】

1) 算法的基本设计思想如下。

分别求两个升序序列 A 、 B 的中位数，设为 a 和 b ，求序列 A 、 B 的中位数过程如下：

- ① 若 $a = b$ ，则 a 或 b 即为所求中位数，算法结束。
- ② 若 $a < b$ ，则舍弃序列 A 中较小的一半，同时舍弃序列 B 中较大的一半，要求两次舍弃的长度相等。
- ③ 若 $a > b$ ，则舍弃序列 A 中较大的一半，同时舍弃序列 B 中较小的一半，要求两次舍弃的长度相等。

在保留的两个升序序列中，重复过程①、②、③，直到两个序列中均只含一个元素时为止，较小者即为所求的中位数。



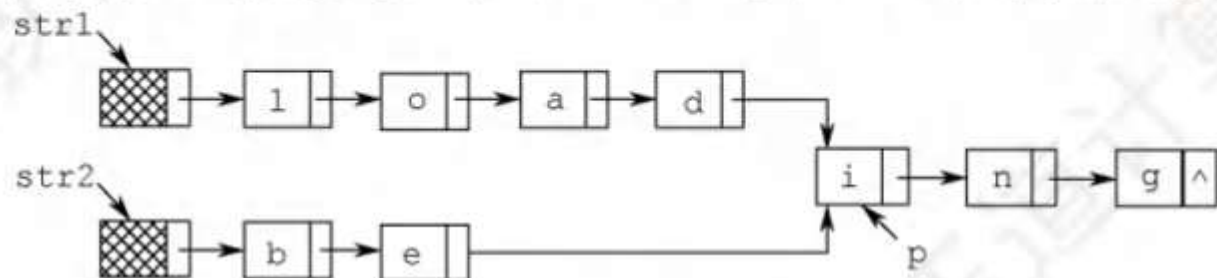
```
int M_Search(int A[],int B[],int n){
    int s1,d1,m1,s2,d2,m2;
    s1=0;d1=n-1;
    s2=1;d2=n-1;
    while(s1!=d1||s2!=d2){
        m1=(s1+d1)/2;
        m2=(s2+d2)/2;
        if(A[m1]==B[m2])
            return A[m1];           //满足条件①
        if(A[m1]<B[m2]){             //满足条件②
            if((s1+d1)%2==0){       //若元素个数为奇数
                s1=m1;              //舍弃 A 中间点以前的部分，且保留中间点
                d2=m2;              //舍弃 B 中间点以后的部分，且保留中间点
            }
            else{                   //元素个数为偶数
                s1=m1+1;            //舍弃 A 的前半部分
                d2=m2;              //舍弃 B 的后半部分
            }
        }
        else{                       //满足条件③
            if((s1+d1)%2==0){       //若元素个数为奇数
                d1=m1;              //舍弃 A 中间点以后的部分，且保留中间点
                s2=m2;              //舍弃 B 中间点以前的部分，且保留中间点
            }
            else{                   //元素个数为偶数
                d1=m1;              //舍弃 A 的后半部分
                s2=m2+1;            //舍弃 B 的前半部分
            }
        }
    }
    return A[s1]<B[s2]? A[s1]:B[s2];
}
```



3) 算法的时间复杂度为 $O(\log_2 n)$, 空间复杂度为 $O(1)$ 。

【另解】对两个长度为 n 的升序序列 A 和 B 中的元素按从小到大的顺序依次访问, 这里访问的含义只是比较序列中两个元素的大小, 并不实现两个序列的合并, 因此空间复杂度为 $O(1)$ 。按照上述规则访问第 n 个元素时, 这个元素即为两个序列 A 和 B 的中位数。

18. 【2012 统考真题】假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，可共享相同的后缀存储空间，例如，loading 和 being 的存储映像如下图所示。



设 `str1` 和 `str2` 分别指向两个单词所在单链表的头结点，链表结点结构为

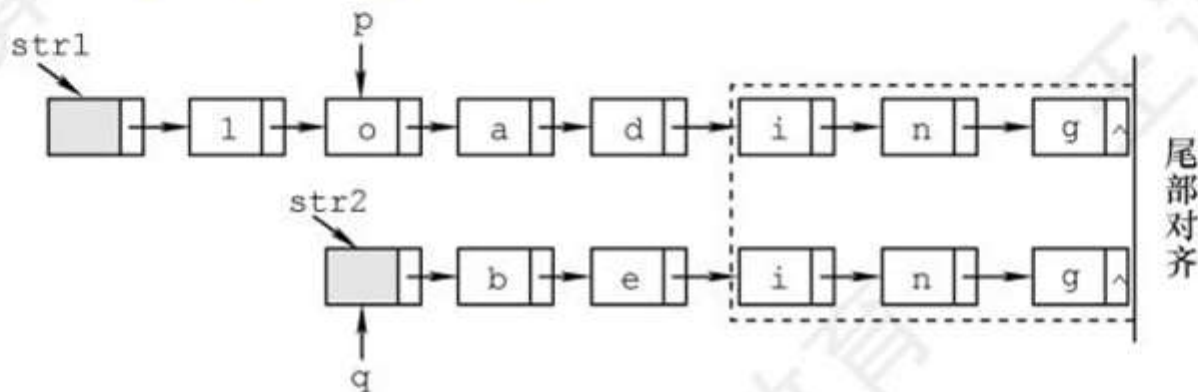
data	next
------	------

，请设计一个时间上尽可能高效的算法，找出由 `str1` 和 `str2` 所指向两个链表共同后缀的起始位置（如图中字符 `i` 所在结点的位置 `p`）。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 或 Java 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度。

18. 【解答】

顺序遍历两个链表到尾结点时，并不能保证两个链表同时到达尾结点。这是因为两个链表的长度不同。假设一个链表比另一个链表长 k 个结点，我们先在长链表上遍历 k 个结点，之后同步遍历两个链表，这样就能够保证它们同时到达最后一个结点。因为两个链表从第一个公共结点到链表的尾结点都是重合的，所以它们肯定同时到达第一个公共结点。



1) 算法的基本设计思想如下:

- ① 分别求出 str1 和 str2 所指的链表的长度 m 和 n 。
- ② 将两个链表以表尾对齐: 令指针 p、q 分别指向 str1 和 str2 的头结点, 若 $m \geq n$, 则指针 p 先走, 使 p 指向链表中的第 $m - n + 1$ 个结点; 若 $m < n$, 则使 q 指向链表中的第 $n - m + 1$ 个结点, 即使指针 p 和 q 所指的结点到表尾的长度相等。
- ③ 反复将指针 p 和 q 同步向后移动, 并判断它们是否指向同一结点。当 p、q 指向同一结点, 则该点即为所求的共同后缀的起始位置。

2) 本题代码如下:

```
typedef struct Node{
    char data;
    struct Node *next;
}SNode;
/*求链表长度的函数*/
int listlen(SNode *head){
    int len=0;
    while(head->next!=NULL){
        len++;
        head=head->next;
    }
    return len;
}
/*找出共同后缀的起始地址*/
SNode* find_list(SNode *str1,SNode *str2){
    int m,n;
    SNode *p,*q;
    m=listlen(str1);           //求 str1 的长度,  $O(m)$ 
    n=listlen(str2);           //求 str2 的长度,  $O(n)$ 
    for(p=str1;m>n;m--)        //若  $m>n$ , 使 p 指向链表中的第  $m-n+1$  个结点
```



```
        p=p->next;
    for (q=str2;m<n;n--)          //若 m<n, 使 q 指向链表中的第 n-m+1 个结点
        q=q->next;
    while (p->next!=NULL&& p->next!=q->next) { //查找共同后缀起始点
        p=p->next;                      //两个指针同步向后移动
        q=q->next;
    }
    return p->next;                //返回共同后缀的起始地址
}
```

3) 时间复杂度为 $O(\text{len1} + \text{len2})$ 或 $O(\max(\text{len1}, \text{len2}))$, 其中 len1 、 len2 分别为两个链表的长度。



12. 【2013 统考真题】已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$). 若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n, 1 \leq k \leq m$), 则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$, 则 5 为主元素; 又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$, 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中, 请设计一个尽可能高效的算法, 找出 A 的主元素。若存在主元素, 则输出该元素; 否则输出 -1。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 或 Java 语言描述算法, 关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。



12. 【解答】

1) 算法的基本设计思想：算法的策略是从前向后扫描数组元素，标记出一个可能成为主元素的元素 Num。然后重新计数，确认 Num 是否是主元素。

算法可分为以下两步：

- ① 选取候选的主元素。依次扫描所给数组中的每个整数，将第一个遇到的整数 Num 保存到 c 中，记录 Num 的出现次数为 1；若遇到的下一个整数仍等于 Num，则计数加 1，否则计数减 1；当计数减到 0 时，将遇到的下一个整数保存到 c 中，计数重新记为 1，开始新一轮计数，即从当前位置开始重复上述过程，直到扫描完全部数组元素。
- ② 判断 c 中元素是否是真正的主元素。再次扫描该数组，统计 c 中元素出现的次数，若大于 $n/2$ ，则为主元素；否则，序列中不存在主元素。

2) 算法实现如下:

```
int Majority(int A[],int n){
    int i,c,count=1;           //c 用来保存候选主元素, count 用来计数
    c=A[0];                   //设置 A[0] 为候选主元素
    for(i=1;i<n;i++)           //查找候选主元素
        if(A[i]==c)
            count++;           //对 A 中的候选主元素计数
        else
            if(count>0)         //处理不是候选主元素的情况
                count--;
            else{               //更换候选主元素, 重新计数
                c=A[i];
                count=1;
            }
    if(count>0)
        for(i=count=0;i<n;i++) //统计候选主元素的实际出现次数
            if(A[i]==c)
                count++;
    if(count>n/2) return c;     //确认候选主元素
    else return -1;            //不存在主元素
}
```

3) 实现的程序的时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

说明: 本题若采用先排序再统计的方法 [时间复杂度为 $O(n\log_2 n)$], 则只要解答正确, 最高可拿 11 分。即便是写出 $O(n^2)$ 的算法, 最高也能拿 10 分, 因此对于统考算法题, 花费大量时间去思考最优解法是得不偿失的。

- 
17. 【2014 统考真题】二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T , 采用二叉链表存储, 结点结构为

left	weight	right
------	--------	-------

其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针, 请设计求 T 的 WPL 的算法, 要求:

- 1) 给出算法的基本设计思想。
- 2) 使用 C 或 C++ 语言, 给出二叉树结点的数据类型定义。
- 3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

17. 【解答】

二叉树的带权路径长度有两种常见的计算方法：①根据二叉树的带权路径长度的定义，二叉树的 WPL 值 = 树中全部叶结点的带权路径长度之和。②根据带权二叉树的性质，二叉树的 WPL 值 = 树中所有非叶结点的权值之和（记住该结论即可，不要求证明）。根据两种常见的计算方法，本题不难写出下列两种解法。

1) 算法的基本设计思想。

① 本问题可采用递归算法实现。根据定义：

$$\begin{aligned}\text{二叉树的 WPL 值} &= \text{树中全部叶结点的带权路径长度之和} \\ &= \text{根结点左子树中全部叶结点的带权路径长度之和} + \\ &\quad \text{根结点右子树中全部叶结点的带权路径长度之和}\end{aligned}$$

叶结点的带权路径长度 = 该结点的 weight 域的值 $\times$ 该结点的深度

设根结点的深度为 0，若某结点的深度为 d 时，则其孩子结点的深度为 $d+1$ 。

在递归遍历二叉树结点的过程中，若遍历到叶结点，则返回该结点的带权路径长度，否则返回其左右子树的带权路径长度之和。

② 若借用非叶结点的 weight 域保存其孩子结点中 weight 域值的和，则树的 WPL 等于树中所有非叶结点 weight 域值之和。

采用后序遍历策略，在遍历二叉树 T 时递归计算每个非叶结点的 weight 域的值，则树 T 的 WPL 等于根结点左子树的 WPL 加上右子树的 WPL，再加上根结点中 weight 域的值。在递归遍历二叉树结点的过程中，若遍历到叶结点，则 return 0 并且退出递归，否则递归计算其左右子树的 WPL 和自身结点的权值。



2) 二叉树结点的数据类型定义如下。

```
typedef struct node{  
    int weight;  
    struct node *left,*right;  
}BTree;
```


3) 算法的代码如下。

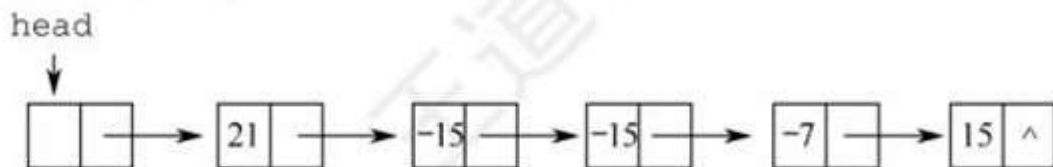
①基于方法 1 的算法实现:

```
int WPL(BTree *root)           //根据 WPL 的定义采用递归算法实现
{
    return WPL1(root,0);
}
int WPL1(BTree *root,int d)    //d 为结点深度
{
    if(root->left==NULL&&root->right==NULL)
        return(root->weight*d);
    else
        return (WPL1(root->left,d+1)+WPL1(root->right,d+1));
}
```

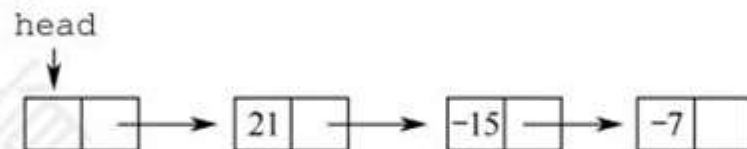
②基于方法 2 的算法实现:

```
int WPL(BTree *root)           //基于递归的后序遍历算法实现
{
    int w_l,w_r;
    if(root->left==NULL&&root->right==NULL)
        return 0;
    else
    {
        w_l=WPL(root->left);    //计算左子树的 WPL
        w_r=WPL(root->right);    //计算右子树的 WPL
        root->weight=root->left->weight+root->right->weight;
                                //填写非叶结点的 weight 域
        return (w_l+w_r+root->weight); //返回 WPL 值
    }
}
```

19. 【2015 统考真题】用单链表保存 m 个整数，结点的结构为 $[data][link]$ ，且 $|data| \leq n$ (n 为正整数)。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 $data$ 的绝对值相等的结点，仅保留第一次出现的结点而删除其余绝对值相等的结点。例如，若给定的单链表 $head$ 如下：



则删除结点后的 $head$ 为



要求：

- 1) 给出算法的基本设计思想。
- 2) 使用 C 或 C++ 语言，给出单链表结点的数据类型定义。
- 3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 4) 说明你所设计算法的时间复杂度和空间复杂度。

19. 【解答】

1) 算法的基本设计思想:

- 算法的核心思想是用空间换时间。使用辅助数组记录链表中已出现的数值,从而只需对链表进行一趟扫描。
- 因为 $|data| \leq n$, 故辅助数组 q 的大小为 $n+1$, 各元素的初值均为 0。依次扫描链表中的各结点, 同时检查 $q[|data|]$ 的值, 若为 0 则保留该结点, 并令 $q[|data|]=1$; 否则将该结点从链表中删除。

2) 使用 C 语言描述的单链表结点的数据类型定义:

```
typedef struct node {  
    int      data;  
    struct node *link;  
}NODE;  
Typedef NODE *PNODE;
```

3) 算法实现如下:

```
void func (PNODE h,int n){
    PNODE p=h,r;
    int *q,m;
    q=(int *)malloc(sizeof(int)*(n+1)); //申请 n+1 个位置的辅助空间
    for(int i=0;i<n+1;i++)                //数组元素初值置 0
        *(q+i)=0;
    while (p->link!=NULL) {
        m=p->link->data>0? p->link->data:-p->link->data;
        if (*(q+m)==0) {                    //判断该结点的 data 是否已出现过
            *(q+m)=1;                        //首次出现
            p=p->link;                        //保留
        }
        else{                               //重复出现
            r=p->link;                        //删除
            p->link=r->link;
            free(r);
        }
    }
    free(q);
}
```

4) 参考答案所给算法的时间复杂度为 $O(m)$, 空间复杂度为 $O(n)$ 。



04. 【2016 统考真题】已知由 n ($n \geq 2$) 个正整数构成的集合 $A = \{a_k | 0 \leq k < n\}$, 将其划分为两个不相交的子集 A_1 和 A_2 , 元素个数分别是 n_1 和 n_2 , A_1 和 A_2 中的元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法, 满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- 3) 说明所设计算法的平均时间复杂度和空间复杂度。

04. 【解答】

1) 算法的基本设计思想

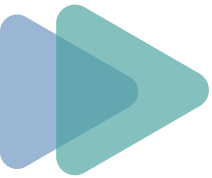
由题意可知，将最小的 $\lfloor n/2 \rfloor$ 个元素放在 A_1 中，其余的元素放在 A_2 中，分组结果即可满足题目要求。仿照快速排序的思想，基于枢轴将 n 个整数划分为两个子集。根据划分后枢轴所处的位置 i 分别处理：

- ① 若 $i = \lfloor n/2 \rfloor$ ，则分组完成，算法结束。
- ② 若 $i < \lfloor n/2 \rfloor$ ，则枢轴及之前的所有元素均属于 A_1 ，继续对 i 之后的元素进行划分。
- ③ 若 $i > \lfloor n/2 \rfloor$ ，则枢轴及之后的所有元素均属于 A_2 ，继续对 i 之前的元素进行划分。

基于该设计思想实现的算法，无须对全部元素进行全排序，其平均时间复杂度是 $O(n)$ ，空间复杂度是 $O(1)$ 。

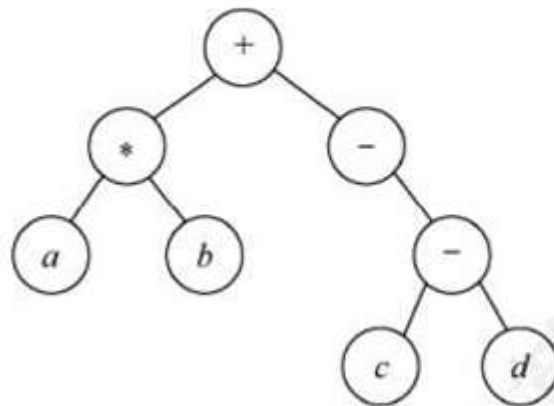
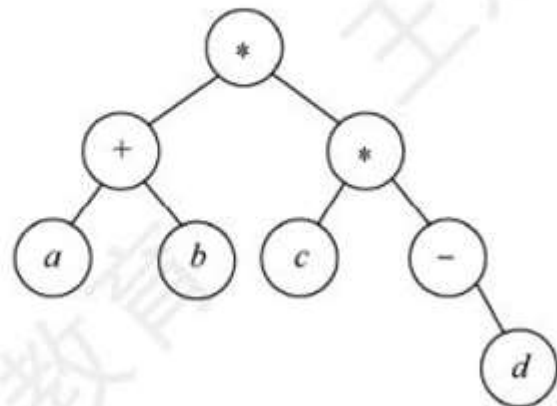
2) 算法实现

```
int setPartition(int a[], int n){
    int pivotkey, low=0, low0=0, high=n-1, high0=n-1, flag=1, k=n/2, i;
    int s1=0, s2=0;
    while(flag) {
        pivotkey=a[low];           //选择枢轴
        while(low<high) {          //基于枢轴对数据进行划分
            while(low<high && a[high]>=pivotkey) --high;
            if(low!=high) a[low]=a[high];
            while(low<high && a[low]<=pivotkey) ++low;
            if(low!=high) a[high]=a[low];
        }                          //end of while(low<high)
        a[low]=pivotkey;
        if(low==k-1)                //若枢轴是第 n/2 小的元素，划分成功
            flag=0;
        else{                       //是否继续划分
            if(low<k-1){
                low0=++low;
                high=high0;
            }
            else{
                high0=--high;
                low=low0;
            }
        }
    }
    for(i=0; i<k; i++) s1+=a[i];
    for(i=k; i<n; i++) s2+=a[i];
    return s2-s1;
}
```



3) 本算法的平均时间复杂度是 $O(n)$ ，空间复杂度是 $O(1)$ 。

18. 【2017 统考真题】请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时：



输出的等价中缀表达式分别为 $(a+b) * (c * (-d))$ 和 $(a*b) + (- (c-d))$ 。

二叉树结点定义如下：

```
typedef struct node{
    char data[10];                //存储操作数或操作符
    struct node *left, *right;
} BTree;
```

要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

18. 【解答】

1) 算法的基本设计思想。

表达式树的中序序列加上必要的括号即为等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。

表达式树中分支结点所对应的子表达式的计算次序，由该分支结点所处的位置决定。为得到正确的中缀表达式，需要在生成遍历序列的同时，在适当位置增加必要的括号。显然，表达式的最外层（对应根结点）和操作数（对应叶结点）不需要添加括号。

2) 算法实现。

将二叉树的中序遍历递归算法稍加改造即可得本题的答案。除根结点和叶结点外，遍历到其他结点时在遍历其左子树之前加上左括号，遍历完右子树后加上右括号。

```
void BtreeToE(BTree *root){
    BtreeToExp(root,1);           //根的高度为 1
}
void BtreeToExp( BTree *root,int deep){
    if(root==NULL) return;        //空结点返回
    else if(root->left==NULL&&root->right==NULL) //若为叶结点
        printf("%s", root->data);    //输出操作数，不加括号
    else{
        if(deep>1) printf("(");      //若有子表达式则加一层括号
        BtreeToExp(root->left, deep+1);
        printf("%s", root->data);    //输出操作符
        BtreeToExp(root->right, deep+1);
        if(deep>1) printf(")");    //若有子表达式则加一层括号
    }
}
```



13. 【2018 统考真题】给定一个含 n ($n \geq 1$) 个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组 $\{-5, 3, 2, 3\}$ 中未出现的最小正整数是 1；数组 $\{1, 2, 3\}$ 中未出现的最小正整数是 4。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

13. 【解答】

1) 算法的基本设计思想:

要求在时间上尽可能高效, 因此采用空间换时间的办法。分配一个用于标记的数组 $B[n]$, 用来记录 A 中是否出现了 $1 \sim n$ 中的正整数, $B[0]$ 对应正整数 1, $B[n-1]$ 对应正整数 n , 初始化 B 中全部为 0。由于 A 中含有 n 个整数, 因此可能返回的值是 $1 \sim n+1$, 当 A 中 n 个数恰好为 $1 \sim n$ 时返回 $n+1$ 。当数组 A 中出现了小于或等于 0 或大于 n 的值时, 会导致 $1 \sim n$ 中出现空余位置, 返回结果必然在 $1 \sim n$ 中, 因此对于 A 中出现了小于或等于 0 或大于 n 的值, 可以不采取任何操作。

经过以上分析可以得出算法流程: 从 $A[0]$ 开始遍历 A , 若 $0 < A[i] \leq n$, 则令 $B[A[i]-1] = 1$; 否则不做操作。对 A 遍历结束后, 开始遍历数组 B , 若能查找到第一个满足 $B[i] == 0$ 的下标 i , 返回 $i+1$ 即为结果, 此时说明 A 中未出现的最小正整数在 1 和 n 之间。若 $B[i]$ 全部不为 0, 返回 $i+1$ (跳出循环时 $i=n$, $i+1$ 等于 $n+1$), 此时说明 A 中未出现的最小正整数是 $n+1$ 。

2) 算法实现:

```
int findMissMin(int A[], int n)
{
```



```
int i,*B; //标记数组
B=(int *)malloc(sizeof(int)*n); //分配空间
memset(B,0,sizeof(int)*n); //赋初值为0
for(i=0;i<n;i++)
    if(A[i]>0&&A[i]<=n) //若 A[i] 的值介于 1~n, 则标记数组 B
        B[A[i]-1]=1;
for(i=0;i<n;i++) //扫描数组 B, 找到目标值
    if (B[i]==0) break;
return i+1; //返回结果
}
```

3) 时间复杂度: 遍历 A 一次, 遍历 B 一次, 两次循环内操作步骤为 $O(1)$ 量级, 因此时间复杂度为 $O(n)$ 。空间复杂度: 额外分配了 $B[n]$, 空间复杂度为 $O(n)$ 。



20. 【2019 统考真题】设线性表 $L = (a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n)$ 采用带头结点的单链表保存，链表中的结点定义如下：

```
typedef struct node
{
    int data;
    struct node*next;
}NODE;
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法，重新排列 L 中的各结点，得到线性表 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \dots)$ 。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计的算法的时间复杂度。



20. 【解答】

1) 算法的基本设计思想

先观察 $L = (a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n)$ 和 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \dots)$, 发现 L' 是由 L 摘取第一个元素, 再摘取倒数第一个元素……依次合并而成的。为了方便链表后半段取元素, 需要先将 L 后半段原地逆置 [题目要求空间复杂度为 $O(1)$, 不能借助栈], 否则每取最后一个结点都需要遍历一次链表。①先找出链表 L 的中间结点, 为此设置两个指针 p 和 q , 指针 p 每次走一步, 指针 q 每次走两步, 当指针 q 到达链尾时, 指针 p 正好在链表的中间结点; ②然后将 L 的后半段结点原地逆置。③从单链表前后两段中依次各取一个结点, 按要求重排。

2) 算法实现

```
void change_list(NODE*h) {  
    NODE *p,*q,*r,*s;  
    p=q=h;  
    while(q->next!=NULL) {           //寻找中间结点  
        p=p->next;                     //p 走一步  
        q=q->next;  
        if(q->next!=NULL) q=q->next;   //q 走两步  
    }  
    q=p->next;                         //p 所指结点为中间结点, q 为后半段链表的首结点  
    p->next=NULL;  
    while(q!=NULL) {                 //将链表后半段逆置  
        r=q->next;  
        q->next=p->next;  
        p->next=q;  
        q=r;  
    }  
    s=h->next;                        //s 指向前半段的第一个数据结点, 即插入点  
    q=p->next;                        //q 指向后半段的第一个数据结点  
    p->next=NULL;  
    while(q!=NULL) {                 //将链表后半段的结点插入到指定位置  
        r=q->next;                    //r 指向后半段的下一个结点  
        q->next=s->next; //将 q 所指结点插入到 s 所指结点之后  
        s->next=q;  
        s=q->next;                    //s 指向前半段的下一个插入点  
        q=r;  
    }  
}
```

3) 第一步找中间结点的时间复杂度为 $O(n)$, 第二步逆置的时间复杂度为 $O(n)$, 第三步合并链表的时间复杂度为 $O(n)$, 所以该算法的时间复杂度为 $O(n)$ 。



14.【2020 统考真题】定义三元组 (a, b, c) (a, b, c 均为整数) 的距离 $D = |a - b| + |b - c| + |c - a|$ 。

给定 3 个非空整数集合 S_1 、 S_2 和 S_3 ，按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法，计算并输出所有可能的三元组 (a, b, c) ($a \in S_1, b \in S_2, c \in S_3$) 中的最小距离。例如 $S_1 = \{-1, 0, 9\}$ ， $S_2 = \{-25, -10, 10, 11\}$ ， $S_3 = \{2, 9, 17, 30, 41\}$ ，则最小距离为 2，相应的三元组为 $(9, 10, 9)$ 。要求：

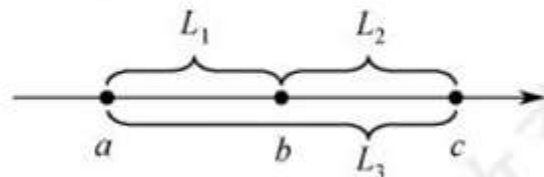
- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 语言或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

14. 【解答】

分析。由 $D = |a - b| + |b - c| + |c - a| \geq 0$ 有如下结论。

① 当 $a = b = c$ 时，距离最小。

② 其余情况。不失一般性，假设 $a \leq b \leq c$ ，观察下面的数轴：



$$L_1 = |a - b|, \quad L_2 = |b - c|, \quad L_3 = |c - a|$$

$$D = |a - b| + |b - c| + |c - a| \geq 0 = L_1 + L_2 + L_3 = 2L_3$$

由 D 的表达式可知，事实上决定 D 大小的关键是 a 和 c 之间的距离，于是问题就可以简化为每次固定 c 找一个 a ，使得 $L_3 = |c - a|$ 最小。

1) 算法的基本设计思想

① 使用 $D_{\min}$ 记录所有已处理的三元组的最小距离，初值为一个足够大的整数。

② 集合 S_1 、 S_2 和 S_3 分别保存在数组 A 、 B 、 C 中。数组的下标变量 $i = j = k = 0$ ，当 $i < |S_1|$ 、 $j < |S_2|$ 且 $k < |S_3|$ 时（ $|S|$ 表示集合 S 中的元素个数），循环执行下面的 a) ~ c)。

a) 计算 $(A[i], B[j], C[k])$ 的距离 D ；（计算 D ）

b) 若 $D < D_{\min}$ ，则 $D_{\min} = D$ ；（更新 D ）

c) 将 $A[i]$ 、 $B[j]$ 、 $C[k]$ 中的最小值的下标+1；（对照分析：最小值为 a ，最大值为 c ，这里 c 不变而更新 a ，试图寻找更小的距离 D ）

③ 输出 $D_{\min}$ ，结束。

2) 算法实现:

```
#define INT_MAX 0x7fffffff
int abs_(int a){//计算绝对值
    if(a<0) return -a;
    else return a;
}
bool xls_min(int a,int b,int c){//a 是否是三个数中的最小值
    if(a<=b&&a<=c) return true;
    return false;
}
int findMinofTrip(int A[],int n,int B[],int m,int C[],int p){
    //D_min 用于记录三元组的最小距离. 初值赋为 INT_MAX
    int i=0,j=0,k=0,D_min=INT_MAX,D;
    while(i<n&&j<m&&k<p&&D_min>0){
        D=abs_(A[i]-B[j])+abs_(B[j]-C[k])+abs_(C[k]-A[i]); //计算 D
        if(D<D_min) D_min=D; //更新 D
        if(xls_min(A[i],B[j],C[k])) i++; //更新 a
        else if(xls_min(B[j],C[k],A[i])) j++;
        else k++;
    }
    return D_min;
}
```

3) 设 $n = (|S_1| + |S_2| + |S_3|)$, 时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

07. 【2021 统考真题】已知无向连通图 G 由顶点集 V 和边集 E 组成, $|E| > 0$, 当 G 中度为奇数的顶点个数为不大于 2 的偶数时, G 存在包含所有边且长度为 $|E|$ 的路径 (称为 EL 路径)。设图 G 采用邻接矩阵存储, 类型定义如下:

```
typedef struct{                                //图的定义
    int  numVertices,numEdges;                //图中实际的顶点数和边数
    char VerticesList[MAXV];                  //顶点表。MAXV 为已定义常量
    int  Edge[MAXV][MAXV];                    //邻接矩阵
}MGraph;
```

请设计算法 `int IsExistEL(MGraph G)`, 判断 G 是否存在 EL 路径, 若存在, 则返回 1, 否则返回 0。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

07.【解答】

1) 算法的基本设计思想

本算法题属于送分题，题干已经告诉我们算法的思想。对于采用邻接矩阵存储的无向图，在邻接矩阵的每一行（列）中，非零元素的个数为本行（列）对应顶点的度。可以依次计算连通图 G 中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

2) 算法实现

```
int IsExistEL(MGraph G) {  
    //采用邻接矩阵存储，判断图是否存在 EL 路径  
    int degree,i,j,count=0;  
    for(i=0;i<G.numVertices;i++){  
        degree=0;  
        for(j=0;j<G.numVertices;j++){  
            degree+=G.Edge[i][j];           //依次计算各个顶点的度  
        }  
        if (degree%2!=0)  
            count++;           //对度为奇数的顶点计数  
    }  
    if (count==0||count==2)  
        return 1;           //存在 EL 路径，返回 1  
    else  
        return 0;           //不存在 EL 路径，返回 0  
}
```

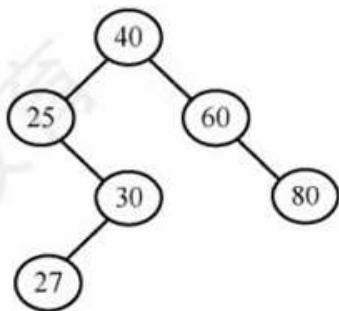
3) 时间复杂度和空间复杂度

算法需要遍历整个邻接矩阵，所以时间复杂度是 $O(n^2)$ ，空间复杂度是 $O(1)$ 。

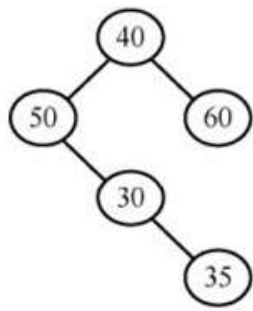
19. 【2022 统考真题】已知非空二叉树 T 的结点值均为正整数，采用顺序存储方式保存，数据结构定义如下：

```
typedef struct {                                //MAX_SIZE 为已定义常量
    int SqBiTNode[MAX_SIZE]; //保存二叉树结点值的数组
    int ElemNum;              //实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如，对于下图所示的两棵非空二叉树 T_1 和 T_2 ，



二叉树 T_1



二叉树 T_2

T_1 的存储结果如下：

T1.SqBiTNode

40	25	60	-1	30	-1	80	-1	-1	27		
----	----	----	----	----	----	----	----	----	----	--	--

T1.ElemNum=10

T_2 的存储结果如下：

T2.SqBiTNode

40	50	60	-1	30	-1	-1	-1	-1	-1	35	
----	----	----	----	----	----	----	----	----	----	----	--

T2.ElemNum=11

请设计一个尽可能高效的算法，判定一棵采用这种方式存储的二叉树是否为二叉搜索树，若是，则返回 true，否则，返回 false。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

19. 【解答 1】

1) 算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在 `SqBiTNode[0]` 中；当某结点保存在 `SqBiTNode[i]` 中时，若有左孩子，则其值保存在 `SqBiTNode[2i+1]` 中；若有右孩子，则其值保存在 `SqBiTNode[2i+2]` 中；若有双亲结点，则其值保存在 `SqBiTNode[(i-1)/2]` 中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。中序遍历二叉搜索树得到一个升序序列。

使用整型变量 `val` 记录中序遍历过程中已遍历结点的最大值，初值为一个负整数。若当前遍历的结点值小于或等于 `val`，则算法返回 `false`，否则，将 `val` 的值更新为当前结点的值。

2) 算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeInOrderBST(SqBiTree bt,int k,int *val){//初始调用时 k 的值是 0
    if(k<bt.ElemNum&&bt.SqBiTNode[k]!=-1){
        if(!judgeInOrderBST(bt,2*k+1,val)) return false;
        if(bt.SqBiTNode[k]<=*val) return false;
        *val=bt.SqBiTNode[k];
        if(!judgeInOrderBST(bt,2*k+2,val)) return false;
    }
    return true;
}
```



【解答 2】

1) 算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在 `SqBiTNode[0]` 中；当某结点保存在 `SqBiTNode[i]` 中时，若有左孩子，则其值保存在 `SqBiTNode[2i+1]` 中；若有右孩子，则其值保存在 `SqBiTNode[2i+2]` 中；若有双亲结点，则其值保存在 `SqBiTNode[(i-1)/2]` 中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。设置两个数组 `pmax` 和 `pmin`。根据二叉搜索树的定义，`SqBiTNode[i]` 中的值应该大于以 `SqBiTNode[2i+1]` 为根的子树中的最大值（保存在 `pmax[2i+1]` 中），小于以 `SqBiTNode[2i+2]` 为根的子树中的最小值（保存在 `pmin[2i+1]` 中）。初始时，用数组 `SqBiTNode` 中前 `ElemNum` 个元素的值对数组 `pmax` 和 `pmin` 初始化。

在数组 `SqBiTNode` 中从后向前扫描，扫描过程中逐一验证结点与子树之间是否满足上述的大小关系。

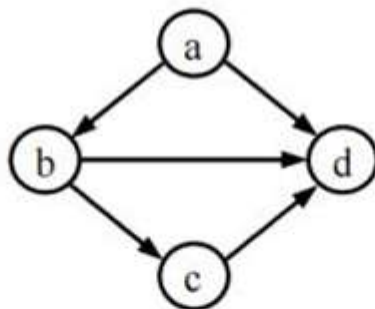
2) 算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeBST(SqBiTree bt){
    int k,m,*pmin,*pmax;
    pmin=(int *)malloc(sizeof(int)*(bt.ElemNum));
    pmax=(int *)malloc(sizeof(int)*(bt.ElemNum));
    for(k=0;k<bt.ElemNum;k++) //辅助数组初始化
        pmin[k]=pmax[k]=bt.SqBiTNode[k];
    for(k=bt.ElemNum-1;k>0;k--) { //从最后一个叶结点向根遍历
        if(bt.SqBiTNode[k]!=-1){
            m=(k-1)/2; //双亲
            if(k%2==1&&bt.SqBiTNode[m]>pmax[k]) //其为左孩子
                pmin[m]=pmin[k];
            else if(k%2==0&&bt.SqBiTNode[m]<pmin[k]) //其为右孩子
                pmax[m]=pmax[k];
            else return false;
        }
    }
    return true;
}
```


08. 【2023 统考真题】已知有向图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct{                                //图的类型定义
    int numVertices,numEdges;                  //图的顶点数和有向边数
    char VerticesList[MAXV];                   //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];                     //邻接矩阵
}MGraph;
```

将图中出度大于入度的顶点称为 K 顶点。例如，在下图中，顶点 a 和 b 为 K 顶点。



请设计算法 `int printVertices(MGraph G)`，对给定的任意非空有向图 G ，输出 G 中所有的 K 顶点，并返回 K 顶点的个数。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

08. 【解答】

1) 算法的基本设计思想:

采用邻接矩阵表示有向图时, 一行中 1 的个数为该行对应顶点的出度, 一列中 1 的个数为该列对应顶点的入度。使用一个初值为零的计数器记录 K 顶点的个数。对图 G 的每个顶点, 根据邻接矩阵计算其出度 outdegree 和入度 indegree。若 $\text{outdegree} - \text{indegree} > 0$, 则输出该顶点且计数器加 1。最后返回计数器的值。

2) 用 C 语言描述的算法:

```
int printVertices (MGraph G) {  
    //采用邻接矩阵存储, 输出 K 顶点, 返回个数  
    int indegree, outdegree, k, m, count=0;  
    for (k=0; k<G.numVertices; k++) {  
        indegree=outdegree=0;  
        for (m=0; m<G.numVertices; m++)    //计算顶点的出度  
            outdegree+=G.Edge[k][m];  
        for (m=0; m<G.numVertices; m++)    //计算顶点的入度  
            indegree+=G.Edge[m][k];  
        if (outdegree>indegree) {  
            printf("%c", G.VerticesList[k]);  
            count++;  
        }  
    }  
    return count;    //返回 K 顶点的个数  
}
```

13. 【2024 统考真题】2023 年 10 月 26 日，神舟十七号载人飞船发射取得圆满成功，再次彰显了中国航天事业的辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为了保证工程的有序开展，需要明确各子工程的前导子工程，以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储，类型定义如下。

```
typedef struct                                //图的类型定义
{
    int numVertices, numEdges;                //图的顶点数和有向边数
    char VerticesList[MAXV];                  //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];                     //邻接矩阵
} MGraph;
```

请设计算法：int uniquely(MGraph G)，判定 G 是否存在唯一的拓扑序列，若是，则返回 1，否则返回 0。要求如下。

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。



13. 【解答】

- 1) 算法的基本设计思想：建立图 G 各顶点的入度表 `degree[]`。选择入度为 0 的顶点 v ，将 v 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 `G.numVertices` 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或者存在多个拓扑序列，返回 0。

2) 用 C 语言描述的算法如下:

```
int uniquely(MGraph G){//判定有向图是否存在唯一的拓扑序列
    int *degree,i,j,count=0,in0=-1,prev_in0;
    degree=(int*)malloc(G.numVertices*sizeof(int));
    for(j=0;j<G.numVertices;j++){//计算各顶点的入度
        degree[j]=0;
        for(i=0;i<G.numVertices;i++)
            degree[j]+=G.Edge[i][j];
        if(degree[j]==0){
            if(in0==-1) in0=j;           //入度为 0 的顶点
            else in0=-2;                //有多个入度为 0 的顶点
        }
    }
    while(in0>=0){
        count++;
        prev_in0=in0;
        in0=-1;
        for(j=0;j<G.numVertices;j++){
            if(G.Edge[prev_in0][j]>0){
                if(--degree[j]==0){      //邻接点入度值减 1
                    if(in0==-1) in0=j;  //入度为 0 的顶点
                    else in0=-2;        //有多个入度为 0 的顶点
                }
            }
        }
    }
    free(degree);
    if(count==G.numVertices) return 1;
    else return 0;
}
```


2487. 从链表中移除节点

算术评级: 4 第 321 场周赛 Q3

1455

🏷 相关标签

🏢 相关企业

💡 提示

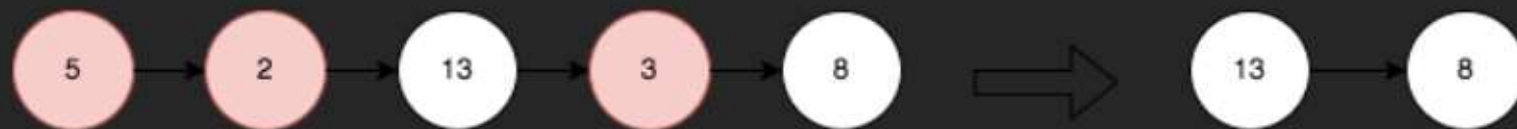
⌘

给你一个链表的头节点 `head`。

移除每个右侧有一个更大数值的节点。

返回修改后链表的头节点 `head`。

示例 1:



输入: `head = [5,2,13,3,8]`

输出: `[13,8]`

解释: 需要移除的节点是 5, 2 和 3。

- 节点 13 在节点 5 右侧。
- 节点 13 在节点 2 右侧。
- 节点 8 在节点 3 右侧。

示例 2:

输入: `head = [1,1,1,1]`

输出: `[1,1,1,1]`

328. 奇偶链表

算术评级: 4

中等

🏷 相关标签

🏢 相关企业

Ax

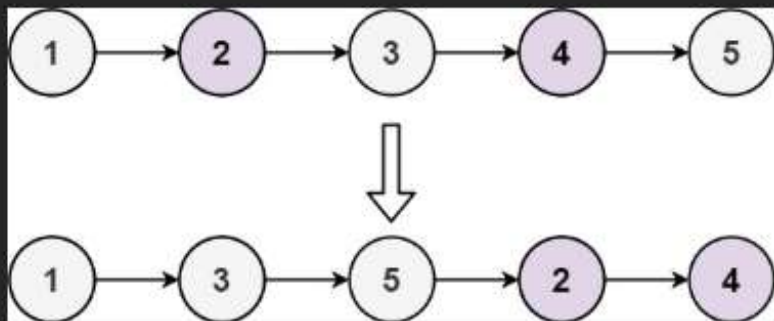
给定单链表的头节点 `head`，将所有索引为奇数的节点和索引为偶数的节点分别分组，保持它们原有的相对顺序，然后把偶数索引节点分组连接到奇数索引节点分组之后，返回重新排序的链表。

第一个节点的索引被认为是奇数，第二个节点的索引为偶数，以此类推。

请注意，偶数组和奇数组内部的相对顺序应该与输入时保持一致。

你必须在 $O(1)$ 的额外空间复杂度和 $O(n)$ 的时间复杂度下解决这个问题。

示例 1:



输入: `head = [1,2,3,4,5]`

输出: `[1,3,5,2,4]`

113. 路径总和 II

算术评级: 4

中等

🔖 相关标签

🏢 相关企业

Ax

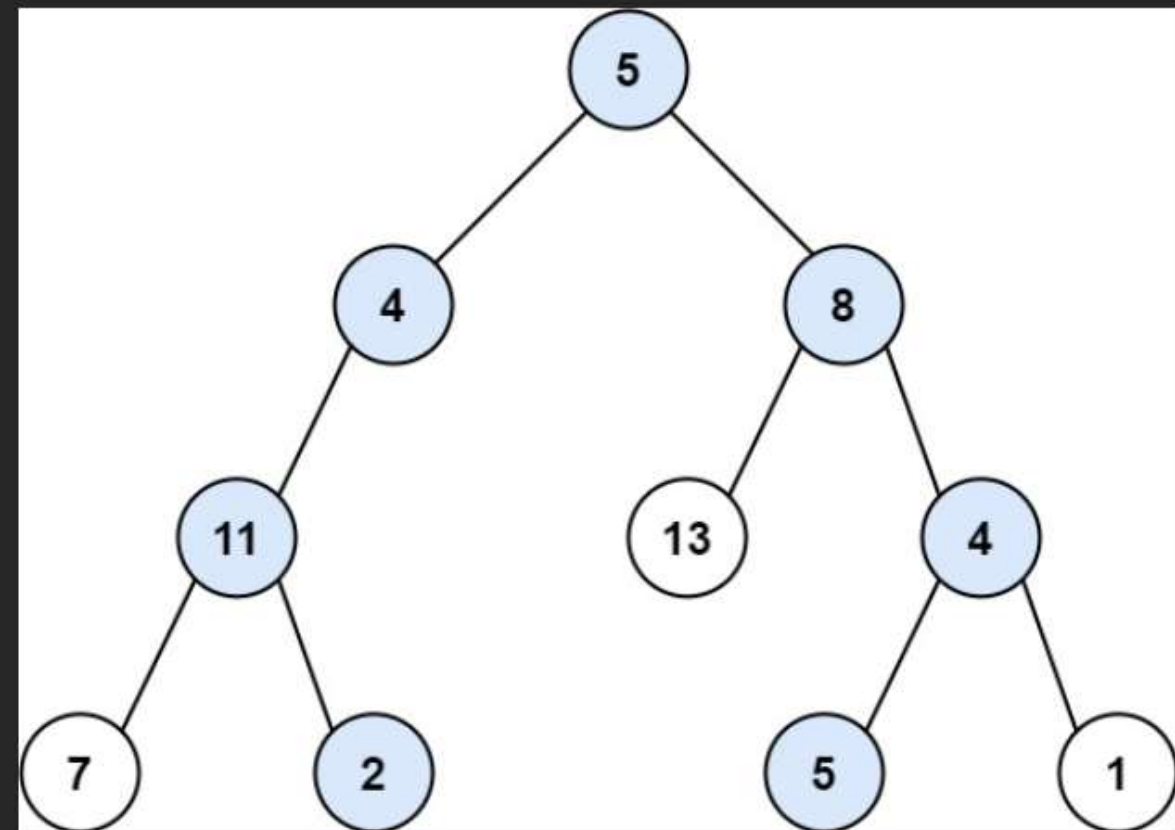
给你二叉树的根节点 `root` 和一个整数目标和 `targetSum`，找出所有 从根节点到叶子节点 路径总和等于给定目标和的路径。

叶子节点 是指没有子节点的节点。

示例 1:

输入: `root = [5,4,8,11,null,13,4,7,2,null,null,5,1]`, `targetSum = 22`

输出: `[[5,4,11,2],[5,8,4,5]]`



513. 找树左下角的值

算术评级: 4

中等

🔖 相关标签

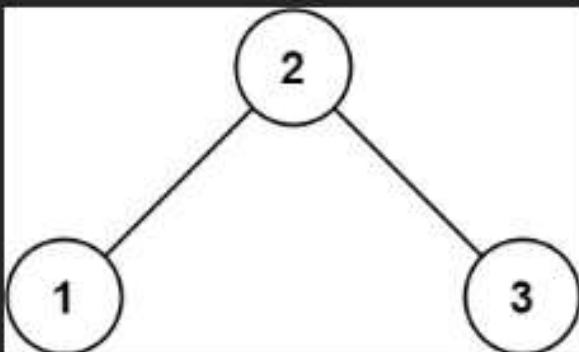
🏢 相关企业

Aa

给定一个二叉树的 **根节点** `root`，请找出该二叉树的 **最底层 最左边** 节点的值。

假设二叉树中至少有一个节点。

示例 1:



输入: `root = [2,1,3]`

输出: 1

814. 二叉树剪枝

算术评级: 4 第 79 场周赛 Q2

1380

🏷 相关标签

🏢 相关企业

Ax

给你二叉树的根结点 `root`，此外树的每个结点的值要么是 `0`，要么是 `1`。

返回移除了所有不包含 `1` 的子树的原二叉树。

节点 `node` 的子树为 `node` 本身加上所有 `node` 的后代。

示例 1:



输入: `root = [1,null,0,0,1]`

输出: `[1,null,0,null,1]`

解释:

只有红色节点满足条件“所有不包含 `1` 的子树”。 右图为返回的答案。

1302. 层数最深叶子节点的和

算术评级: 4 第 16 场双周赛 Q3

1388

🏷 相关标签

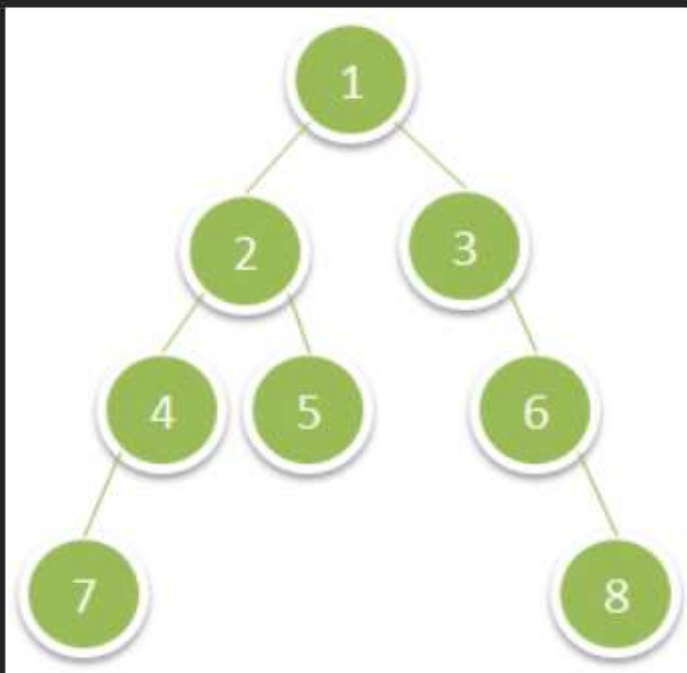
🏢 相关企业

💡 提示

Ax

给你一棵二叉树的根节点 `root`，请你返回 层数最深的叶子节点的和。

示例 1:



输入: `root = [1,2,3,4,5,null,6,7,null,null,null,null,8]`

输出: 15

547. 省份数量

算术评级: 4

中等

🏷 相关标签

🏢 相关企业

Aa

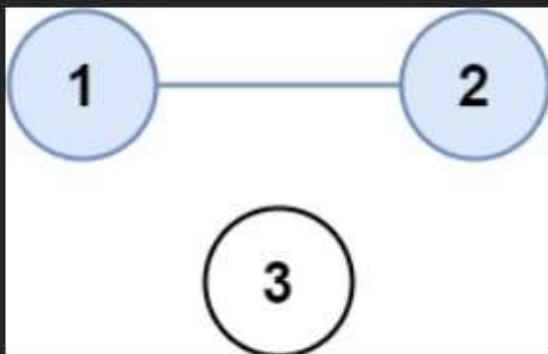
有 n 个城市，其中一些彼此相连，另一些没有相连。如果城市 a 与城市 b 直接相连，且城市 b 与城市 c 直接相连，那么城市 a 与城市 c 间接相连。

省份 是一组直接或间接相连的城市，组内不含其他没有相连的城市。

给你一个 $n \times n$ 的矩阵 `isConnected`，其中 `isConnected[i][j] = 1` 表示第 i 个城市和第 j 个城市直接相连，而 `isConnected[i][j] = 0` 表示二者不直接相连。

返回矩阵中 **省份** 的数量。

示例 1:



输入: `isConnected = [[1,1,0],[1,1,0],[0,0,1]]`

输出: 2

1557. 可以到达所有点的最少点数目

算术评级: 4 第 33 场双周赛 Q2

1512

🏷 相关标签

🏢 相关企业

💡 提示

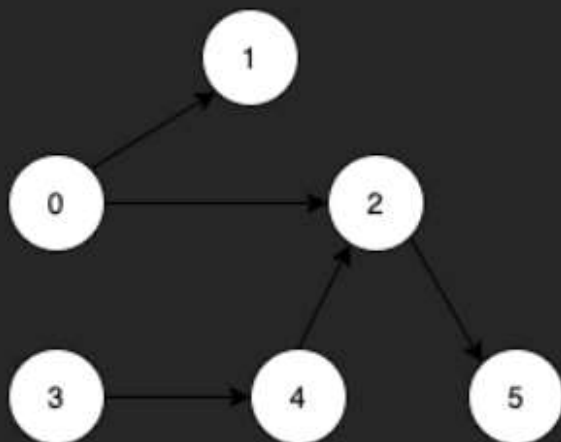
Ax

给你一个有向无环图， n 个节点编号为 0 到 $n-1$ ，以及一个边数组 `edges`，其中 `edges[i] = [fromi, toi]` 表示一条从点 `fromi` 到点 `toi` 的有向边。

找到最小的点集使得从这些点出发能到达图中所有点。题目保证解存在且唯一。

你可以以任意顺序返回这些节点编号。

示例 1:



输入: $n = 6$, `edges = [[0,1],[0,2],[2,5],[3,4],[4,2]]`

输出: `[0,3]`

解释: 从单个节点出发无法到达所有节点。从 0 出发我们可以到达 `[0,1,2,5]`。从 3 出发我们可以到达 `[3,4,2,5]`。所以我

743. 网络延迟时间

算术评级: 5

中等

🔍 相关标签

🏢 相关企业

💡 提示

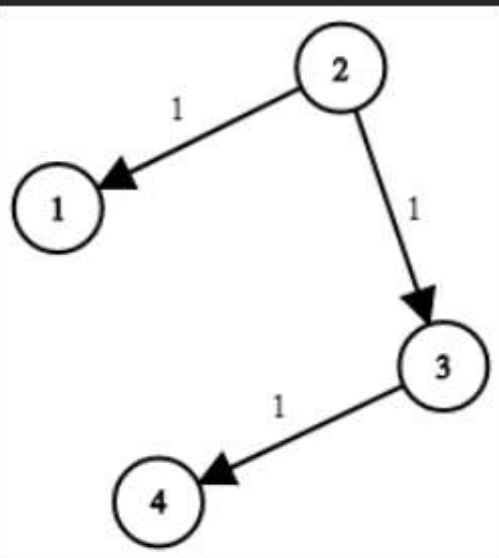
Ax

有 n 个网络节点，标记为 1 到 n 。

给你一个列表 `times`，表示信号经过 有向 边的传递时间。 $\text{times}[i] = (u_i, v_i, w_i)$ ，其中 u_i 是源节点， v_i 是目标节点， w_i 是一个信号从源节点传递到目标节点的时间。

现在，从某个节点 k 发出一个信号。需要多久才能使所有节点都收到信号？如果不能使所有节点收到信号，返回 -1 。

示例 1:



输入: `times = [[2,1,1],[2,3,1],[3,4,1]]`, $n = 4$, $k = 2$

输出: 2

653. 两数之和 IV - 输入二叉搜索树

算术评级: 3

简单

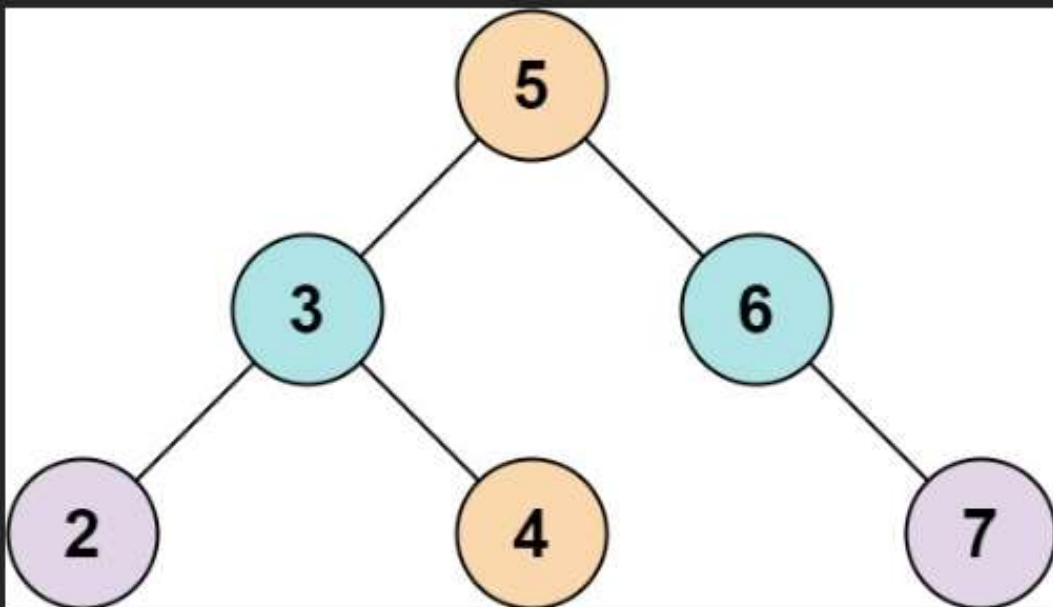
🔍 相关标签

🏢 相关企业

Ax

给定一个二叉搜索树 `root` 和一个目标结果 `k`，如果二叉搜索树中存在两个元素且它们的和等于给定的目标结果，则返回 `true`。

示例 1:



输入: `root = [5,3,6,2,4,null,7]`, `k = 9`

输出: `true`